

VotaSJ: A Blockchain-Anchored Participatory Budgeting Platform for São José/SC

Davi Ludvig Longen Machado¹, Lucas Furlanetto Pascoali¹
Arthur Clasen de Melo¹, Pedro Henrique Tesman Mansani da Silva¹

¹Departamento de Informática e Estatística (INE)
Universidade Federal de Santa Catarina (UFSC)
Florianópolis — SC — Brasil

{davi.ludvig,lucas.furlanetto,arthur.clasen,pedro.tesman}@grad.ufsc.br

Abstract. *Municipal participatory budgeting (PB) in Brazil suffers from three structural failures: turnout below 2 % of the eligible population, operational cost in the order of hundreds of thousands of reais per cycle, and a tally that the City Hall both runs and announces — so every contested result ends in an unverifiable accusation of fraud. We propose VotaSJ, a permissioned-write, permissionless-read application on Polygon PoS that uses gov.br for eligibility, smart contracts for cycle state and tally, IPFS for proposal metadata, and an on-chain milestone log for execution accountability. Administration is held by a 3-of-5 multisig (City Hall $\times$ 3, UFSC, TCE-SC) and contract upgrades are gated by a 14-day timelock. This paper argues the narrow technical case for blockchain in this domain, details the contract design (implementation, 40-test Hardhat suite), reports gas measurements ($\approx$ 430k gas per full citizen flow), and discusses the threat model, LGPD posture, and migration path to a Semaphore-based vote-secrecy expansion. We do not claim that blockchain “solves democracy”; we claim it is the right primitive for the narrow set of steps where operator honesty cannot be assumed.*

1. Introduction and Executive Summary

Brazilian municipal participatory budgeting (PB) has been around since the 1990s, but in cities the size of São José/SC it rarely works in practice. Turnout sits under 2 % of eligible voters, a single cycle costs the city hundreds of thousands of reais in logistics, and the City Hall both runs the count and announces the winner — which means every contested result ends in an accusation of fraud that nobody can prove or disprove. The institution survives mostly as political theatre.

VotaSJ replaces the parts of that institution where a public ledger genuinely helps — the eligibility commitment, the vote, the tally, and the milestone log of how the winning proposal is executed — and leaves everything else off-chain. Citizens authenticate with gov.br [Governo Digital 2026], the federal digital ID; the backend computes a salted credential hash and commits it to Polygon PoS [Polygon Labs 2026] through a VoterRegistry; the ParticipatoryBudget cycle contract handles proposals, voting, and tally; an ExecutionTracker records contract signing, work start, milestones, and completion of the winning proposal, with IPFS-anchored evidence. Administration sits on a 3-of-5 Safe (City Hall $\times$ 3, UFSC $\times$ 1, TCE-SC $\times$ 1), and any contract upgrade

is gated by a 14-day `OpenZeppelin TimelockController` [OpenZeppelin 2026], so the rules cannot change inside an active cycle.

Why blockchain at all? Because no single party in this system is trusted by all the others. Citizens distrust the City Hall to count its own vote; the opposition distrusts the incumbent; auditors distrust the speed of paper reconstruction. What the system has to produce is a count that nobody has to take on faith, and a public L2 with an external validator set produces that property without anyone in São José having to be honest. Sections 2–3 make that case; 4–8 describe the system; 9–11 cover security, privacy and scale; 12–13 report what it costs; 14–15 introduce the team and go-to-market plan; 16–18 close with roadmap, risks, and conclusion.

What VotaSJ is **not**: an attempt to “tokenize democracy” or to replace deliberation with code. Citizens pay nothing. There is no fungible vote-token. Deliberation, mobilization, and assembly happen off-chain where they belong. The blockchain is just the record of who is allowed to vote, who voted for what, and what happened to the money — the four things the institution currently cannot prove.

2. Problem Statement

São José/SC has approximately **180,000 eligible voters** and runs (irregularly) a participatory-budget process inherited from the Porto Alegre school of the early 1990s, framed legally by the *Estatuto da Cidade* [Brasil 2001]. Field data from comparable mid-size Brazilian municipalities indicates:

- **Turnout below 2 %** in the last three documented cycles — under 3,600 actual participants decide the allocation of tens of millions of reais of investment-margin budget.
- **Operational cost per cycle of BRL 300k–500k**, dominated by physical assembly logistics, poll-worker payments, manual tallying, and administrative reconciliation.
- **A tally that takes ≈ 3 weeks** of paper reconstruction after the last assembly closes, during which the legitimacy of the result is essentially indefensible.
- **No execution feedback loop**: even when a proposal wins, citizens have no programmatic way to verify whether the committed budget was actually executed against the winning project.

Three additional second-order effects compound the failure: (i) *self-selection bias* because evening assemblies systematically exclude shift workers, parents of small children, and the geographically peripheral population; (ii) *fraud asymmetry*, since the City Hall operates, counts, and announces the result; and (iii) *bureaucratic opacity post-vote*, since the winning proposal disappears into the municipal financial system (SIG) after the cycle closes.

2.1. Why existing digital systems fail

Since roughly 2015 Brazilian municipalities have experimented with digital PB tools — from off-the-shelf survey platforms (Decidim, Consul, SurveyMonkey) to in-house web portals integrated with municipal SSO. These deployments share four failure modes:

- **Single-operator trust**. The City Hall owns the database, the application server, and the announcement. A determined operator can modify results without detectable trace.

- **No public auditability.** Citizens get an HTML page; auditors get a CSV upon request weeks later. Neither version is independently verifiable against the original event stream because the event stream is, in principle, a SQL table the operator can rewrite.
- **No Sybil resistance.** Most deployments fall back to CPF + email confirmation, which is not Sybil-resistant.
- **No execution accountability.** None of the deployed systems link the vote result back to budget execution.

VotaSJ does not claim that blockchain solves all four problems by default. It claims, more narrowly, that **blockchain solves the first two structurally**; that the third is solvable by anchoring identity to `gov.br`, an existing federal credential infrastructure the operator does not control; and that the fourth is solvable by reusing the same substrate that records the vote to record the milestone.

3. Why Blockchain (and Why That Is Not a Tautology)

Before settling on a blockchain we considered three simpler alternatives: a normal web application backed by a relational database, the same setup with a cryptographic audit log on top (Merkle roots, signed receipts), and a PKI-only design where each voter holds a certificate and votes are detached signatures. Each one fixes part of the problem and leaves another part open.

3.1. Traditional web servers and relational databases

This is what every existing Brazilian deployment uses. Integrity depends entirely on operator honesty; the auditor has access only to what the operator chooses to expose. Adequate for *non-political* civic systems; unfit for one whose entire value proposition is convincing a skeptical opposition that the count is honest.

3.2. Centralized DB with cryptographic audit log

The operator publishes Merkle roots of the event stream and citizens receive signed receipts. A meaningful improvement over §3.1, and the strongest non-blockchain candidate (the architecture behind Certificate Transparency). The auditor can prove *inclusion* and *exclusion* of a given receipt against the published root, but **cannot prove completeness** — there may be additional events the operator chose not to commit. The “completeness” property is exactly the one a political adversary will attack.

3.3. PKI-only systems

Every voter holds a certificate; votes are detached signatures. Per-vote integrity is excellent, but *aggregation integrity* is nil — the aggregator still decides what counts as the final tally. Operational reality also dooms it: governmental PKI rollout (e.g., ICP-Brasil) has been notoriously expensive and slow.

3.4. Public blockchain

Integrity is enforced by a consensus protocol on a validator set the operator does not control [Wood 2014]; auditability is trivial; completeness is by construction; live verification is continuous with ≈ 2 s finality on Polygon PoS. A blockchain is the only candidate in this list where the integrity guarantee does not collapse if the operator becomes adversarial.

3.5. On-chain only where it matters

Subproblem	On-chain?	Why
Proposal metadata (title, images)	no	IPFS; commit only the CID.
Personal data (CPF, name, address)	no	LGPD [Brasil 2018]; never on-chain.
Proving eligibility	partial	gov.br proof, hash on-chain.
Casting a vote	yes	Operator honesty cannot be assumed.
Counting votes	yes	A <code>closeCycle()</code> on-chain is the only count nobody must trust.
Recording the winner	yes	Same.
Tracking budget execution	yes	The accountability loop the institution fails to close.
Notifications, PDFs, charts	no	Standard off-chain web.

4. System Architecture

VotaSJ is a three-layer system. Only the on-chain layer is authoritative; the off-chain backbone is a best-effort cache and integration; the citizen layer is the UX and the audit surface.

+-----+
L1 - Citizen layer
* Progressive Web App (React + ethers v6)
* Assisted kiosks in CRAS / UPAs / public libraries
* Public audit portal (read-only subgraph queries)
+-----+
L2 - Off-chain backbone
* Voter Registry DB (PostgreSQL, LGPD-controlled)
* gov.br OIDC relay (Serpro federated identity)
* IPFS pinning (proposal metadata)
* The Graph subgraph (public indexer)
* Chainlink Automation (cycle-closing keeper)
* Notification worker (push/email/SMS - non-authoritative)
+-----+
L3 - On-chain substrate of record (Polygon PoS)
* VoterRegistry.sol
* ParticipatoryBudget.sol (cycle, propose, vote, close)
* ExecutionTracker.sol (milestone commitments)
* TimelockController + 3-of-5 Safe multisig
* UUPS proxy pattern (upgradeable behind timelock)
+-----+

Listing 1. High-level architecture

4.1. Why Polygon PoS, not Ethereum L1, not a private chain

Ethereum L1 is the natural conservative choice but at current gas prices it produces a per-vote cost in the order of USD 0.20–1.00, which over 50,000 votes per cycle becomes prohibitive without buying additional security relative to a major L2. A private/permissioned chain (e.g., Hyperledger Fabric) fails the central test: the operator (City Hall) would also control the validator set — the chain degenerates into the configuration ruled out in §3.2. A roll-your-own validator set (UFSC + TCE-SC + Public Prosecutor’s Office running their own nodes) is plausible but requires significant institutional negotiation and is fragile against political reconfiguration; it is kept as a deliberate fallback. Polygon PoS gives us a public L2 with ≈ 100 external validators, ≈ 2 s finality, EVM equivalence, and per-tx fees in fractions of a cent [Polygon Labs 2026].

5. Smart Contract Design

VotaSJ splits the on-chain state into two contracts with deliberately narrow responsibilities — a `VoterRegistry` and a `ParticipatoryBudget` cycle machine — plus an `ExecutionTracker` for milestone commitments. The contract suite is exercised by a 40-test Hardhat suite [Nomic Foundation 2026].

5.1. VoterRegistry

Maintains the set of eligible voter addresses keyed by a salted credential hash. Admin-only writes (`registerVoter`, `revokeVoter`), public reads. The registry is small (one address per citizen) and rarely changes.

5.2. ParticipatoryBudget

A finite-state cycle machine with three states (Pending, Open, Closed). Public writes are gated on `registry.isRegistered(msg.sender)`; admin-only writes (`openCycle`) are gated on the multisig. The implementation is wrapped behind a UUPS proxy [OpenZeppelin 2026].

State transitions:

```
Pending --openCycle()--> Open
Open --closeCycle() after closesAt--> Closed
Closed --openCycle()--> Open (next cycle)
```

Three invariants are enforced at the contract level and exercised in the test suite:

1. Only registered voters can submit a proposal or cast a vote.
2. One vote per voter per cycle.
3. No cycle reopens once closed.

Tally is deterministic: `closeCycle()` iterates the proposals of the current cycle and emits `CycleClosed(cycleId, winningProposalId, winningVotes)`. There is no off-chain tally service to trust.

5.3. ExecutionTracker

Once a cycle closes with a winning proposal, the City Hall financial system (SIG) consumes the on-chain event via a webhook and triggers the off-chain execution flow. As budget is committed and disbursed, milestone commitments are written back via `ExecutionTracker.recordMilestone(proposalId, milestoneId, cidEvidence)` by a multisig-controlled relay, with IPFS-anchored evidence (invoices, photographs, contract scans). This is the *accountability loop* that the existing institution fails to close.

5.4. Storage layout and gas economy

The packing decisions are deliberate: `opensAt` and `closesAt` are `uint64` (sufficient until year 292277026596), letting the EVM pack them into a single storage slot, while larger fields land in their own slots. Empirical gas measurements from the Hardhat test suite indicate roughly $\approx 430,000$ gas for a full citizen flow (register + submit + vote + read). At a Polygon gas price of 30 gwei and a MATIC price of $\approx$ USD 0.40, the per-citizen-per-cycle cost lands at $\approx$ USD 0.005 — three orders of magnitude below the existing process cost.

5.5. Events as the indexing surface

We design events as a first-class API, not a debugging afterthought: `CycleOpened`, `ProposalSubmitted`, `VoteCast`, `CycleClosed`. The public subgraph [The Graph Foundation 2026] is built strictly from these events; the audit portal queries the subgraph. Citizens, journalists, TCE-SC, and the Public Prosecutor's Office consume the same surface — no one gets a privileged audit channel because no one needs one.

5.6. Upgradeability (UUPS + Timelock)

The production deployment wraps `ParticipatoryBudget` behind a UUPS proxy. The implementation contract is the only target of `upgradeTo`, which is restricted to the multisig and gated by a 14-day `TimelockController` [OpenZeppelin 2026]. UUPS over Transparent Proxy reduces per-call gas overhead by $\approx 2,000$ gas; the 14-day timelock is calibrated to be longer than the critical voting interval. A second guardrail is enforced at the application level: `openCycle()` reverts if a pending upgrade is queued. The combination makes “rewrite-the-tally-rules-during-the-vote” a strict impossibility under the multisig threat model.

5.7. Implementation notes for the next developer

This subsection collects the design decisions that are not obvious from the contract source. We document them here because each is a place where a future maintainer is likely to second-guess us.

- **Revert error strings follow a "Contract: reason" convention** ("Budget: not admin", "Budget: cycle not open", "Registry: zero address"). The frontend pattern-matches on the prefix to route errors to the right UI surface; renaming a contract changes the prefix and is a backward-incompatible UI change.

- **opensAt and closesAt are uint64** (sufficient through year 292277026596) so they pack into a single storage slot with `status` and `proposalCount`. Using `uint256` everywhere would cost an extra slot per cycle and $\approx 3,000$ gas per `openCycle` for no practical benefit.
- **The VoterRegistry reference is immutable** in `ParticipatoryBudget`. The registry address is fixed at construction; rebinding to a different registry would require redeploying the budget contract and migrating cycle history. This is deliberate: a swappable registry is a larger attack surface than a redeployment is a maintenance cost.
- **Single active cycle invariant.** `openCycle` requires the previous cycle to be `Closed`. Multi-cycle parallelism (one per administrative region) is a roadmap item.
- **closeCycle iterates linearly** over `proposalCount` to find the winner. We bound `proposalCount` at 500 per cycle; above that the close transaction approaches Polygon’s 30M block gas limit. For larger deployments, the tally moves to an off-chain ZK rollup with a single on-chain verification — that work is scoped but not yet built.
- **Vote receipts are events, not return values.** Callers reconstruct receipts from the `VoteCast` event, which lets the subgraph be the single source of truth for “did my vote land”.
- **No comments, ratings, or rich proposal interaction on-chain.** Anything that could carry personal data, opinion text, or moderation surface stays in IPFS or in the off-chain backend; the contract knows only proposal IDs, vote counts, and milestone hashes.

6. Identity Verification Flow

VotaSJ does not authenticate citizens directly. It delegates to `gov.br` [Governo Digital 2026], the Brazilian federal OIDC identity provider operated by Serpro:

1. The citizen opens the VotaSJ PWA and clicks “Register to vote”.
2. The PWA redirects to `gov.br` with `scope=openid profile govbr-confianca-alto`.
3. `gov.br` returns a signed assertion containing the citizen’s CPF and confidence level.
4. The backend computes `credentialHash = keccak256(cpf || municipality_code || cycle_salt || pepper)`, where `pepper` is a server-side secret rotated every cycle. The CPF never leaves the boundary; only the hash does.
5. The backend asks the citizen to bind a wallet address (in-app ERC-4337 [Buterin et al. 2024] smart-account, or external).
6. The wallet $\leftrightarrow$ `credentialHash` binding is committed via `VoterRegistry.registerVoter` from the multisig.
7. From this point, the citizen votes from the wallet. The chain knows nothing about the CPF.

The pepper-and-salt construction means the same CPF produces a different `credentialHash` each cycle, which prevents long-term linkability of a voter across

cycles by anyone who later obtains the database. The pepper is held by the City Hall under multisig escrow; it is never required to be known publicly to verify the integrity of the count.

Trust assumption. We assume gov.br is honest about the identities it asserts. This is the same assumption every federal service makes; we do not improve it, but we also do not weaken it. Mitigation against a gov.br compromise is institutional, not cryptographic — a credential-hash anomaly (e.g., a sudden burst of registrations from a single IP block) is detectable by the auditor multisig and triggers a pause.

7. Multisig Governance

Administrative authority is held by a **3-of-5 Gnosis Safe**:

Signer	Role
City Hall #1 (Planning Secretariat)	Executive
City Hall #2 (Ombudsman)	Internal control
City Hall #3 (Legal Department)	Procedural validity
UFSC INE / Blockchain Lab	Academic auditor
TCE-SC (State Court of Audit)	Institutional auditor

The choice is deliberate. A pure City Hall multisig would not solve the political-asymmetry problem of §2. A purely institutional multisig would make routine operations slow and politically fraught. The 3-of-5 with three internal and two external signers, threshold 3, encodes:

- The City Hall **cannot** unilaterally open, close, or modify a cycle (needs at least one external signer).
- The City Hall **can** still run routine operations (3 internal signatures) when no auditor objects.
- Any single external signer can veto by withholding their signature.

The Safe is itself wrapped by an OpenZeppelin `TimelockController` [OpenZeppelin 2026] for any operation that changes contract code, role assignments, or the signer set. Routine cycle operations are not timelocked — they happen at the speed of multisig coordination.

8. Data Flow and Processing

End-to-end, the lifecycle of a single vote:

1. Citizen authenticates with gov.br [Governo Digital 2026].
2. Backend issues `credentialHash`; multisig calls `VoterRegistry.registerVoter`.
3. Citizen browses proposals; metadata loaded from IPFS, addressed by the on-chain CID.
4. Citizen calls `ParticipatoryBudget.vote(proposalId)` via an ERC-4337 [Buterin et al. 2024] paymaster funded by the City Hall — the citizen pays no gas.
5. Polygon validators include the transaction; finality in ≈ 2 s.

6. The subgraph indexer [The Graph Foundation 2026] ingests the `VoteCast` event.
7. The public audit portal updates the per-proposal count in real time.
8. After `closesAt`, Chainlink Automation [Chainlink Labs 2026] calls `closeCycle()`; tally is deterministic.
9. The winning proposal is read by the City Hall’s SIG via webhook.
10. As execution progresses, milestones are committed to `ExecutionTracker`.

The citizen sees a normal mobile UX; the auditor sees a public event log; the City Hall sees a SIG integration. Nobody is forced to interact with the blockchain directly except as the system-of-record.

9. Security Model and Threat Analysis

9.1. Threat actors and assets

Actor	Capability	Asset at risk
Adversarial City Hall	3/5 signers, runs backend	Tally outcome
External attacker	Internet, backend compromise	Voter Registry, gas budget
Vote buyer	Money, social pressure	Vote secrecy
Sybil attacker	Multiple CPFs	Vote weight
Polygon validator collusion	$\geq 2/3$ validators	Liveness, finality
Smart-contract bug	—	Cycle integrity

9.2. Sybil resistance

The chain itself does not provide Sybil resistance. We obtain it externally via gov.br: each CPF produces only one `credentialHash` per cycle, and each `credentialHash` is bound to at most one address by the registry. The remaining vector — one human, multiple CPFs — requires either a stolen identity (separately a crime, detectable by gov.br’s anti-fraud telemetry) or a corrupted gov.br assertion (federal-level compromise, out of scope). We explicitly do **not** rely on captcha, phone-number verification, or email confirmation for Sybil resistance — none of those are Sybil-resistant at scale.

9.3. Admin abuse resistance

Three layers: (1) critical state changes require 3-of-5 signatures including ≥ 1 external signer; (2) critical *rule* changes additionally require a 14-day timelock; (3) `openCycle` reverts if any contract upgrade is pending — closing the loophole of “upgrade now, execute against the new rules immediately”.

9.4. Censorship resistance

The contract is deployed on a public L2 with ≈ 100 external validators. The City Hall cannot unilaterally take VotaSJ offline; doing so would require either shutting down Polygon (which they do not control) or pushing a contract upgrade to disable voting (timelocked, multisig-gated, publicly visible).

9.5. Smart-contract attack surface

We treat the smart contract as the single highest-value target:

- External audit (OpenZeppelin or Hacken) before every major release, budgeted at BRL 40k–80k per audit.
- Reentrancy protection via `ReentrancyGuard` [OpenZeppelin 2026] on any state-mutating function that touches an external contract.
- Integer overflow safety automatic in Solidity ≥ 0.8 .
- Pausable escape hatch wired to the multisig.
- No `delegatecall` outside the UUPS proxy slot.
- Slither + Mythril static analysis in CI on every PR.

9.6. Emergency procedure

If the multisig detects an active attack (Sybil wave, smart-contract bug, Polygon-level reorg): (1) any 3-of-5 signers call `pause()`; (2) the Public Prosecutor's Office and the citizenry are notified via the public portal banner; (3) diagnosis is off-chain; the cycle is closed and re-opened with new parameters via multisig + timelock, or allowed to expire and its result discarded with public, signed justification.

10. Privacy and LGPD Compliance

LGPD [Brasil 2018] is binding on the City Hall as data controller. VotaSJ's posture:

- **No personal data on-chain.** Ever. CPF, full name, address, phone, email — all live in the off-chain Voter Registry DB.
- **Only credential hashes on-chain.** One-way function; unrecoverable without the per-cycle pepper.
- **Per-cycle pepper rotation.** Long-term linkability across cycles is prevented at the chain level.
- **Right-to-be-forgotten (LGPD Art. 18).** The on-chain `credentialHash` cannot be deleted but, because it is a one-way function of a CPF the system no longer holds, the on-chain residue is, in our reading, non-identifying. This interpretation has not yet been confirmed by the ANPD for our specific deployment.
- **DPO and Data Processing Agreement.** The City Hall's appointed DPO co-signs the LGPD impact assessment.

10.1. Future zk-proof eligibility (Semaphore expansion)

The current registry-based design exposes one cryptographic linkage that the next iteration breaks: an external observer who controls the Voter Registry DB and observes a `VoteCast` event can correlate the wallet address with the citizen. This is not a leak to the *public* (the wallet is pseudonymous) but it is one to the *City Hall*. The next iteration replaces the `registry.isRegistered(msg.sender)` check with a Semaphore-style group-membership zk-proof [Privacy and Scaling Explorations, Ethereum Foundation 2026, Semaphore Protocol contributors 2026]:

- The eligible cohort is committed as a Merkle root of credential hashes (one root per cycle).

- The citizen generates a zk-SNARK proving “I know a leaf in this tree and I have not voted before” without revealing which leaf.
- A nullifier deterministic in `(credentialHash, cycleId)` is published on-chain to enforce one-vote-per-citizen without linkability to the registry.

Benchmarks of comparable Semaphore-on-Polygon deployments suggest a per-vote zk-proof generation of $\approx 3\text{--}5$ s on a mid-range mobile device and an on-chain verification cost of $\approx 250\text{k}$ gas, still within the per-vote economic envelope. The zk migration is scheduled for the second annual cycle.

11. Scalability, Performance, and Infrastructure

11.1. Throughput envelope

Polygon PoS sustains ≈ 65 TPS of complex EVM transactions and $\approx 1\text{--}2$ kTPS of simple ones, with finality in the ≈ 2 s range [Polygon Labs 2026]. A peak voting day under the VotaSJ cycle profile ($\approx 20,000$ citizens voting in the last 48 h of a 14-day cycle) implies a peak sustained load of ≈ 0.1 TPS — three orders of magnitude below the chain’s capacity.

11.2. Gas-cost considerations

Operation	Gas (approx.)
<code>registerVoter</code>	80k
<code>submitProposal</code>	120k
<code>vote</code>	95k
<code>closeCycle</code> (per proposal)	30k

At 30 gwei and MATIC $\approx$ USD 0.40, the per-vote cost is on the order of USD 0.001. Even with a $10\times$ safety margin and ERC-4337 paymaster overhead [Buterin et al. 2024], the per-cycle gas budget at 50,000 voters fits comfortably under BRL 5,000.

11.3. `closeCycle` loop bound

The current `closeCycle` implementation iterates over all proposals. We bound the maximum number of proposals per cycle to 500. At $500 \times 30\text{k}$ gas, the close transaction lands at $\approx 15\text{M}$ gas — well under Polygon’s 30M block gas limit. For larger municipalities, the next iteration will move the tally to an off-chain ZK-rollup with on-chain verification.

11.4. Infrastructure requirements

- Dedicated Polygon node (Erigon, 8 vCPU / 32 GB RAM / 4 TB SSD), with a redundant hot standby.
- IPFS pinning via a managed provider (Pinata or local Kubo cluster) with 10 GB of pinned content per cycle and 99.9 % availability SLA.
- The Graph subgraph on the decentralized network [The Graph Foundation 2026] with a secondary self-hosted indexer.
- Chainlink Automation subscription, funded once per cycle [Chainlink Labs 2026].

11.5. Reliability and fault tolerance

Three independent failure scopes:

Scope	Failure	Mitigation
Polygon	Reorg, multi-block halt	Multisig pause; cycle replay; ultimate fallback to Ethereum L1 re-deployment (contracts are EVM-portable).
Backend	gov.br outage, IPFS unpin, indexer lag	Retries; alternative IPFS gateway; subgraph re-sync.
Operator	City Hall non-cooperation	The chain keeps running; 2 external signers; any RPC works.

The trust-critical path (cast vote, count vote, publish result) only depends on Polygon being live. Every other failure mode degrades UX, not integrity.

12. Operational Workflow

A canonical cycle, end-to-end:

```
T-90 City Hall + UFSC + TCE-SC agree on cycle parameters.
T-60 Multisig queues openCycle() in TimelockController (14-day
    delay).
T-46 openCycle() executes. Cycle is Pending.
T-30 Proposal-submission window opens.
T-16 Proposal window closes; voting window opens.
T-2 Last 48h: peak voting load.
T0 closesAt reached. Chainlink Automation calls closeCycle().
T+0 CycleClosed event emitted; portal shows winner.
T+1 City Hall SIG receives webhook; budget execution kicks off.
T+30/60/90 ExecutionTracker.recordMilestone() with IPFS evidence.

T+365 Next cycle: GOTO T-90.
```

No human is required to perform a count, publish a result, or open a notification thread; all are produced as side-effects of on-chain events.

12.1. Operational runbook (who does what during a cycle)

The contract suite removes humans from the count, but the cycle still requires named operators with explicit handoffs. We document them here so the team running the first municipal pilot has a checklist rather than a vibe.

Stage	Owner	Activity
Cycle config	Planning Secretariat	Drafts cycle params; circulates to UFSC + TCE-SC for review.
Timelock queue	Secretariat + ≥ 1 external signer	Queues <code>openCycle</code> in the <code>TimelockController</code> .
Mobilization	Comms + neighborhood associations	PWA, press, kiosks; trains kiosk attendants.
Identity onboarding	Backend on-call	Operates the gov.br relay; monitors registration anomalies.
Voting window	Backend on-call + multisig	Monitors gas, paymaster, Polygon block production.
Close	Chainlink Automation	Calls <code>closeCycle()</code> at <code>closesAt</code> .
Result publication	Frontend + comms	Portal updates from subgraph; press release triggered by <code>CycleClosed</code> .
Execution tracking	Secretariat + multisig	<code>ExecutionTracker.recordMilestone()</code> with IPFS evidence at each stage.

The single biggest operational risk is **paymaster underfunding** during the last 48 hours of voting: at 30 gwei and the gas profile of \$11, the paymaster needs roughly BRL 2,000–3,000 of MATIC held in advance to cover 50k votes with a $5\times$ safety margin. We pre-fund the paymaster two weeks before the voting window and check the balance daily during the cycle.

The second biggest risk is a **multisig signer being unreachable** in an emergency. The 3-of-5 threshold gives one signer of slack, but only one. The signer set has a documented escalation chain kept off-chain in the operations handbook.

13. Economic Feasibility

13.1. Cost reduction vs. traditional process

Item	Traditional	VotaSJ
Assembly logistics	60–100k	0
Poll workers, transport	80–150k	5–10k
Manual tallying	40–80k	0
Administrative reconciliation	30–50k	0
Communications, mobilization	50–80k	40–60k
External smart-contract audit (amortized)	—	40–80k
Gas + infrastructure	—	5–10k
Total (BRL/cycle)	260–460k	90–160k

Net savings: BRL 170k–300k per cycle, conservatively, before counting the political value of an unimpeachable result.

13.2. Revenue model

VotaSJ is a B2G SaaS sold to the City Hall: annual subscription BRL 180–250k, per-cycle setup fee BRL 40k, implementation consulting for additional municipalities BRL 60–120k.

Citizens never pay.

14. Team

VotaSJ was designed and implemented by four senior students of the **Departamento de Informática e Estatística (INE)** at UFSC, enrolled in **INE5458 — Blockchain and Cryptocurrency Technologies** (2026/1). Each author contributed across the full stack; the primary specializations are summarized below.

Member	Primary contribution
Davi Ludvig Longen Machado	Smart-contract architecture: cycle state machine, UUPS-proxy and multisig wiring, OpenZeppelin integration, Hardhat test harness.
Lucas Furlanetto Pascoali	Smart-contract security: threat modeling, Slither/Mythril CI gates, gas profiling, reentrancy and access-control review.
Arthur Clasen de Melo	Off-chain backbone: gov.br OIDC relay design, IPFS pinning topology, The Graph subgraph schema, Chainlink Automation upkeep.
Pedro H. T. Mansani da Silva	Citizen-facing layer: PWA (React + ethers v6), ERC-4337-based wallet abstraction [Buterin et al. 2024], accessibility and kiosk flows.

The team writes Solidity, TypeScript, and Python daily; all four members have prior Brazilian-civic-tech exposure through coursework, hackathons, and contributions to UFSC research projects. The work is positioned within the long-running research line of the **UFSC Blockchain Lab** at INE, which provides academic supervision and acts as the proposed external auditor signer of the production multisig. External smart-contract auditing for the first municipal pilot is contracted to a specialized firm (OpenZeppelin or Hacken) before any binding deployment — the team explicitly does not self-audit security-critical code.

15. Go-to-Market and Marketing Strategy

VotaSJ is a B2G product. Its commercial cycle is slow, reference-driven, and institutionally gated — not a consumer-acquisition campaign. The marketing plan reflects that reality and is built on top of the *Channels* and *Customer Relationships* blocks of the Business Model Canvas.

15.1. Sales motion (long-cycle B2G)

- **Anchor sale.** A municipal contract with São José/SC as the reference deployment. In Brazilian municipal procurement, the first signature is the gating asset.
- **Reference-based expansion.** From São José, the next addressable cluster is Greater Florianópolis (Palhoça, Biguaçu, Florianópolis), followed by mid-size Santa Catarina municipalities via **FECAM** (Federação Catarinense de Municípios) and **ABM** (Associação Brasileira de Municípios) channels.

- **Institutional dual-use.** TCE-SC and the Public Prosecutor’s Office function simultaneously as auditor multisig signers *and* as the strongest endorsement signal a municipal buyer can receive. Their participation in governance is itself the marketing message.

15.2. Communication channels (mapped from the BMC)

Channel	Audience	Type
VotaSJ PWA & audit portal	Citizens, journalists	Own digital
The Graph subgraph + REST API	TCE-SC, MP, academia	Own M2M
gov.br OI DC integration page	Citizens (onboarding)	Partner digital
Assisted kiosks at CRAS/UPAs	Unbanked, elderly	Own physical
Neighborhood associations	Grassroots citizens	Partner grassroots
Regional press (NDTV/SC, ND)	All citizens	Partner media
Conferences (SBSeg, SBRC, LADC)	Auditors, researchers	Own academic
Open-source repository	Engineers, municipal IT	Own technical

15.3. Content artifacts

The white-paper series itself is the spine of the strategy. Three artifacts, deliberately layered: (1) this **long technical paper** for engineers, CTOs, TCE-SC technical staff, and academic reviewers; (2) a **short business/user paper** (companion deliverable, leaflet style) for elected officials and citizens; and (3) a **pitch deck** and **demo video** for funding-oriented audiences. The artifacts are cross-referenced and versioned; updates are published on the open-source repository so that journalists and auditors can always cite a stable, dated source.

15.4. Pilot-driven activation

The first municipal cycle is the marketing event of the year. Kiosks are co-located in CRAS/UPAs/libraries — venues citizens already attend for unrelated reasons — and staffed by city-hall social-services personnel. The cycle produces three by-products of equal commercial value: (a) measured turnout (the only metric a competitor cannot fake); (b) photographic evidence of execution milestones (the substance of the “accountability loop” value proposition); and (c) press-ready stories of named citizens whose votes turned into named municipal works.

16. Future Roadmap

Phase	Target	Scope
Delivered	Contract suite + spec	VoterRegistry, ParticipatoryBudget and ExecutionTracker implemented, 40-test Hardhat suite, gas envelope characterized.
2026-Q3	Field pilot	Single cycle in one administrative region of São José, 5–10k participants.
2026-Q4	Full São José cycle	Citywide, ≈50k voters.
2027	Multi-municipality	Onboarding Palhoça, Biguaçu, Florianópolis.
2027–2028	zk-Semaphore default	Full vote-secrecy from City Hall, not just from the public [Privacy and Scaling Explorations, Ethereum Foundation 2022].
2028+	Polygon zkEVM migration	Preserves contract semantics; EVM-portable by design.

17. Risks, Limitations, and Ethical Considerations

What VotaSJ does **not** solve:

- It does not produce deliberation. A digital ballot is not an assembly.
- It does not solve the digital divide. Roughly 8 % of São José adults do not own a smartphone; assisted kiosks mitigate but do not eliminate the exclusion.
- It is not a coup-proof system. A determined adversary controlling 3-of-5 signers and willing to wait 14 days can rewrite the rules.
- Polygon is a dependency. If Polygon ceases to be credible, migration is non-trivial (though tractable, given EVM portability).
- Legal feasibility of binding digital voting under the São José Organic Law is not formally established. The first field pilot is consultative, not binding, pending an OAB-SC and municipal legal opinion.

A few constraints we treat as non-negotiable. They are listed here because some of them rule out otherwise reasonable engineering shortcuts.

- **Citizens pay nothing — money, gas, or cognitive overhead.** The day a citizen is asked to hold MATIC, install a wallet, or understand the word “blockchain” to vote, the system loses its constituency. The ERC-4337 [Buterin et al. 2024] paymaster, the in-app wallet abstraction, and the gov.br SSO are there to keep that promise, not to be impressive.
- **No tradeable representation of voting power.** No fungible vote-token, no transferable SBT for voting rights, no anything that could spawn a secondary market. Participation badges are non-transferable (ERC-5192-style SBT [Daubenschütz and Anders 2022, Ohlhaver et al. 2022]), per-cycle, wallet-bound.

- **Pseudonymity, not anonymity — and we say so.** Today’s votes link wallet $\leftrightarrow$ `credentialHash`, opaque to the public but visible to the City Hall via the registry. The second annual cycle moves to Semaphore-based anonymity. The paper does not claim a property the system does not yet enforce.
- **Source available, audit available.** Contracts, indexer schema, and audit reports are public. No “secret sauce”.
- **No surveillance side-channel.** Push notifications, analytics, and UX telemetry are opt-in and routed through the off-chain backend; none of them can influence the count.

18. Conclusion

Two things are true at once about Brazilian participatory budgeting: it works on paper (the law backs it, the budgets exist) and it fails in practice (almost nobody votes, nobody trusts the count). The failure is not a missing dashboard or a slow website — it is the political fact that the City Hall both runs the process and announces the winner, with no neutral party in between. A blockchain does not fix democracy, but it does fix that particular asymmetry: a public L2 with an external validator set [Polygon Labs 2026] produces a count that nobody in São José has to take on faith.

VotaSJ uses that property as narrowly as possible. Eligibility, votes, tally, and execution milestones live on-chain; everything else stays where it belongs — `gov.br` [Governo Digital 2026] for identity, IPFS for proposal content, a normal web backend for UX. The 3-of-5 multisig with two external signers and the 14-day time-lock [OpenZeppelin 2026] make it impossible for any single party, including the City Hall itself, to change the rules during an active vote. The per-cycle cost lands at roughly one-third of the existing process. The citizen never has to know what a blockchain is.

What remains open is institutional, not technical: a legal opinion on the bindingness of digital voting under the São José Organic Law, an ANPD position on hash-anchored eligibility, and the operational readiness of the City Hall as the anchor customer. The pilot is the next step; the contracts, the threat model, and the cost envelope are already in place.

We started this work because every cycle in the existing system survives on the goodwill of organizers and the patience of a small constituency, and we have watched it lose legitimacy to fraud claims that nobody can rebut. The point of VotaSJ is to make the count something a citizen can check from the bus home — and to make the City Hall something a citizen can hold to a deadline. The smallest amount of blockchain that fixes those two things is the contribution of this paper.

References

- [Brasil 2001] Brasil (2001). Lei nº 10.257, de 10 de julho de 2001 (Estatuto da Cidade). https://www.planalto.gov.br/ccivil_03/leis/leis_2001/110257.htm. Accessed 2026-05.
- [Brasil 2018] Brasil (2018). Lei nº 13.709, de 14 de agosto de 2018 (Lei Geral de Proteção de Dados Pessoais). https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/113709.htm. Accessed 2026-05.

- [Buterin et al. 2024] Buterin, V., Weiss, Y., Tirosh, D., Nacson, S., Forshtat, A., Gazso, K., and Hess, T. (2024). ERC-4337: Account abstraction using alt mempool. Ethereum Improvement Proposals, <https://eips.ethereum.org/EIPS/eip-4337>. Accessed 2026-05.
- [Chainlink Labs 2026] Chainlink Labs (2026). Chainlink automation documentation. <https://docs.chain.link/chainlink-automation>. Accessed 2026-05.
- [Daubenschütz and Anders 2022] Daubenschütz, T. and Anders (2022). ERC-5192: Minimal soulbound NFTs. Ethereum Improvement Proposals, <https://eips.ethereum.org/EIPS/eip-5192>. Accessed 2026-05.
- [Governo Digital 2026] Governo Digital (2026). Roteiro de integração do login único gov.br. <https://acesso.gov.br/roteiro-tecnico/>. Accessed 2026-05.
- [Nomic Foundation 2026] Nomic Foundation (2026). Hardhat documentation. <https://hardhat.org/docs>. Accessed 2026-05.
- [Ohlhaver et al. 2022] Ohlhaver, P., Weyl, E. G., and Buterin, V. (2022). Decentralized society: Finding Web3’s soul. SSRN Working Paper 4105763, https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4105763. Accessed 2026-05.
- [OpenZeppelin 2026] OpenZeppelin (2026). OpenZeppelin contracts 5.x documentation. <https://docs.openzeppelin.com/contracts/5.x/>. Accessed 2026-05.
- [Polygon Labs 2026] Polygon Labs (2026). Polygon PoS documentation. <https://docs.polygon.technology/>. Accessed 2026-05.
- [Privacy and Scaling Explorations, Ethereum Foundation 2026] Privacy and Scaling Explorations, Ethereum Foundation (2026). Semaphore protocol documentation. <https://docs.semaphore.pse.dev/>. Accessed 2026-05.
- [Semaphore Protocol contributors 2026] Semaphore Protocol contributors (2026). semaphore-protocol/semaphore (GitHub repository). <https://github.com/semaphore-protocol/semaphore>. Accessed 2026-05.
- [The Graph Foundation 2026] The Graph Foundation (2026). The graph documentation. <https://thegraph.com/docs/en/>. Accessed 2026-05.
- [Wood 2014] Wood, G. (2014). Ethereum: A secure decentralised generalised transaction ledger. <https://ethereum.github.io/yellowpaper/paper.pdf>. Yellow Paper, latest revision; accessed 2026-05.